

UNIVERSIDAD TÉCNICA DEL NORTE

Facultad de Ingeniería en Ciencias Aplicadas · Ingeniería de Software

Proyecto Integrador — UTN GolMundial 2026

Plan de reparto de trabajo del equipo (4 integrantes)

Desarrollo · Modelamiento UML (PowerDesigner) · Procesos (AuraPortal) · Wireframes (Balsamiq) · Documentación

1. Asignación general de componentes

El equipo está conformado por cuatro integrantes. Cada uno lidera un componente del sistema, respetando la regla de arquitectura del proyecto: los dos backends usan tecnologías distintas (Jakarta EE y .NET) y los dos frontends también (JSF y C#), con una base de datos independiente por backend. El liderazgo implica responsabilidad sobre el componente, no exclusividad: todos programan en todos los componentes.

Integrante	Componente	Tecnología	Base de datos	Diagramas UML
Alexis	Backend Estadísticas	Jakarta EE (JAX-RS + JPA / WildFly)	PostgreSQL (Linux)	Ninguno
Jonathan	Backend UTNGolCoin	ASP.NET Core Web API (C#)	MySQL	4
Fer	Frontend Administrativo	Blazor / ASP.NET Core MVC (C#)	—	4
Dayana	Frontend Público	JSF + PrimeFaces	—	4

Alexis no elabora diagramas UML; a cambio asume mayor carga de procesos, despliegue, contrato REST común, pruebas de integración y capítulos adicionales del informe, de modo que la carga total quede equilibrada entre los cuatro.

2. ALEXIS — Backend Servicio de Estadísticas

Jakarta EE + PostgreSQL · Despliegue obligatorio en Linux · Sin diagramas UML

Responsabilidades técnicas	Backend de Estadísticas en Jakarta EE (JAX-RS + JPA/Hibernate sobre WildFly). Carga del seed (48 selecciones, 12 grupos, 104 partidos, sedes USA/México/Canadá). Endpoints de consulta (calendario, grupos, posiciones, estadísticas, detalle de partido, fases). Cálculo automático de posiciones (RF06). Registro de resultado y notificación REST a UTNGolCoin (RF12). Autenticación centralizada con hash (RNF04) y auditoría. Swagger (RNF09). Despliegue obligatorio en Linux (RNF07). Degradación controlada (RNF05). Definición y versionado del contrato REST común (OpenAPI). Líder del despliegue final en Linux de ambos backends.
Módulos asignados	Calendario · Tablas de posiciones · Estadísticas por selección · Gestión del torneo (dominio) · Autenticación (backend centralizado) · Auditoría (backend) · Fases eliminatorias.
Requisitos	RF04, RF05, RF06, RF07, RF08, RF09, RF12, RF24, RF28. RNF04, RNF05, RNF06, RNF07, RNF08, RNF09.
Historias de usuario	HU02, HU03, HU06 (backend: registrar resultado → actualizar posiciones y disparar liquidación).
Procesos y subprocesos (AuraPortal)	Carga inicial del seed del torneo · Registro de resultado oficial y actualización de posiciones · Autenticación centralizada y gestión de sesiones (RNF04) · Gestión de fases eliminatorias y avance del torneo (RF09) · Subproceso: Notificación de resultado a UTNGolCoin (RF12) · Subproceso: Registro de acciones administrativas / auditoría (RF24).
Diagramas UML (PowerDesigner)	NINGUNO (por decisión del equipo). Aporta a los demás la topología de arquitectura y los contratos REST para asegurar la coherencia de sus diagramas.
Interfaces Frontend (Balsamiq)	Ninguna (backend). Entrega los contratos REST que consumen ambos frontends.
Capítulos del informe	2.6 Definiciones, siglas y abreviaturas (tecnologías backend) · 3.2 Base de datos → Servicio de Estadísticas (usuarios, roles, selecciones, grupos, sedes, partidos, auditoría) · 3.3 Tablas de Estadísticas · 4.1 Perspectiva del producto (arquitectura + despliegue Linux) · 4.2.1 Estructura y funciones del Servicio de Estadísticas · 4.2.3.2 Dependencias · 7. Referencias (fuentes técnicas Jakarta EE, REST, PostgreSQL, Linux).
Rol transversal / QA	Despliegue final en Linux (ambos backends) · contrato REST común versionado · pruebas de degradación controlada (RNF05). Ejecuta CP04, CP09 y CP13.
Entregables	API Estadísticas + Swagger · seed cargado en PostgreSQL · evidencia de despliegue en Linux · contrato REST común · ADR (backend/BD).
Dependencias	Es la dependencia raíz del proyecto: Jonathan depende de su notificación (RF12), identificador de usuario y autenticación; Fer y Dayana dependen de sus endpoints de consulta. Debe entregar el contrato REST temprano (Sprint 1–2).

3. JONATHAN — Backend Servicio UTNGolCoin

ASP.NET Core Web API (C#) + MySQL

Responsabilidades técnicas	Backend UTNGolCoin en ASP.NET Core Web API (C#) sobre MySQL. Billeteras, ledger inmutable de transacciones, predicciones 1X2 (validación de saldo RF16, única por partido RF18, cierre por hora RF17), liquidación automática monto × cuota (RF19), bono de bienvenida de 10 como transacción (RF01), bono anti-bancarrota 1/día (RF20), ranking (RF21), reportes de circulación (RF27). Endpoint receptor de la notificación de resultado. Swagger (RNF09). Despliegue .NET en Linux opcional (+puntos).
Módulos asignados	Predicciones UTNGolCoin · Billetera · Ranking de usuarios · Reportes (circulación de monedas) · Bono de bienvenida y bono diario.
Requisitos	RF01, RF13, RF14, RF15, RF16, RF17, RF18, RF19, RF20, RF21, RF22, RF27. RNF04, RNF05, RNF06, RNF08, RNF09.
Historias de usuario	HU01, HU04, HU05, HU07, HU08, HU09.
Procesos y subprocesos (AuraPortal)	Registro de predicción 1X2 (validación de saldo y cierre automático) · Liquidación automática de premios (RF19) · Subproceso: Acreditación del bono de bienvenida (RF01) · Subproceso: Bono diario anti-bancarrota (RF20) · Subproceso: Generación de reportes de circulación (RF27).
Diagramas UML (PowerDesigner)	Sequence · Communication · Statechart · Use Case (vista global de actores admin/usuario/invitado).
Interfaces Frontend (Balsamiq)	Ninguna (backend). Provee el contrato REST de predicciones, billetera y ranking que consume el Frontend Público.
Capítulos del informe	2.6 Definiciones (moneda virtual, ledger, gamificación, .NET/MySQL) · 3.2 Base de datos → Servicio UTNGolCoin (billeteras, transacciones, predicciones, bonos_diarios) · 3.3 Tablas de UTNGolCoin · 4.2.2 Estructura y funciones del Servicio UTNGolCoin.
Rol transversal / QA	Ejecuta los casos de prueba de su dominio: CP03, CP05, CP06, CP07, CP08, CP10, CP11, CP12.
Entregables	API UTNGolCoin + Swagger · MySQL con ledger · liquidación automática operativa · ADR (backend/BD) · Diagramas de secuencia, comunicación, estados y casos de uso.
Dependencias	Depende de Alexis (notificación RF12, identificador de usuario, autenticación) y de su contrato REST. Para el Use Case depende de Fer y Dayana (casos de uso concretos de cada interfaz). Dayana depende de sus endpoints de predicción/billetera/ranking.

4. FER — Frontend Administrativo

Blazor / ASP.NET Core MVC (C#) · Diseño sobrio tipo panel de control

Responsabilidades técnicas	Frontend Administrativo en Blazor / ASP.NET Core MVC (C#), diseño sobrio tipo panel de control (RNF02), responsive con Adaptive Theme (RNF01). Consume las APIs vía REST/JSON. Gestión de partidos, registro de resultados oficiales (UI), gestión de usuarios/roles, reportes y auditoría (UI). Validación en frontend (RNF03) y mensajes claros (RNF10). Wireframes y mockups en Balsamiq de sus pantallas.
Módulos asignados	Autenticación/registro (admin) · Gestión del torneo (UI admin) · Gestión de usuarios y roles · Reportes y auditoría (UI).
Requisitos	RF10, RF11 (UI registro de resultado), RF23, RF24 (UI auditoría), RF25, RF27 (UI reportes). RNF01, RNF02, RNF03, RNF08, RNF10.
Historias de usuario	HU06 (UI registrar resultado), HU10 (gestión de usuarios y roles).
Procesos y subprocesos (AuraPortal)	Gestión de partidos y selecciones (RF10) · Registro de resultado oficial desde el panel (RF11, UI) · Gestión de usuarios y roles (RF23, RF25) · Subproceso: Consulta de reportes y auditoría (RF24, RF27, UI). Consolida además el proceso global integrador end-to-end.
Diagramas UML (PowerDesigner)	Deployment · Component · Package · Composite Structure.
Interfaces Frontend (Balsamiq)	Diseña e implementa las mismas: login admin, panel de control, gestión de partidos, formulario de registro de resultado, gestión de usuarios/roles, pantalla de reportes/auditoría.
Capítulos del informe	2.1 Problema · 2.2 Justificación · 2.3 Objetivos · 2.4 Propósitos · 2.5 Alcance · 2.7 Personal Involucrado · 4.2.3.1 Restricciones de diseño · 4.4 Costos (con Jonathan).
Rol transversal / QA	Lidera la integración de servicios y las pruebas de aceptación. Consolida el proceso end-to-end en AuraPortal. Ejecuta CP01, CP02, CP09, CP14.
Entregables	Frontend Administrativo funcional (diseño sobrio) · mockups Balsamiq admin · ADR del frontend C# · Diagramas de despliegue, componentes, paquetes y estructura compuesta · plan de pruebas de integración.
Dependencias	Depende de endpoints de Alexis (gestión/resultados) y Jonathan (reportes de circulación). Para Deployment/Component necesita la topología de Alexis y la estructura del servicio de Jonathan. Coordina el Use Case general con Jonathan y Dayana.

5. DAYANA — Frontend Público

JSF + PrimeFaces · Diseño llamativo con temática mundialista

Responsabilidades técnicas	Frontend Público en JSF (Jakarta Faces / PrimeFaces), diseño llamativo con temática mundialista (RNF02), responsive con Adaptive Theme (RNF01). Consume ambas APIs vía REST/JSON. Registro/login/logout e invitado, calendario, posiciones, estadísticas, predicciones, billetera y ranking. Validación en frontend (RNF03) y mensajes claros (RNF10). Wireframes y mockups en Balsamiq de sus pantallas.
Módulos asignados	Autenticación/registro (público, UI) · Calendario (UI) · Tablas de posiciones (UI) · Estadísticas por selección (UI) · Predicciones UTNGolCoin (UI) · Billetera (UI) · Ranking (UI).
Requisitos	RF01–RF03 (UI registro/login/logout), RF04–RF09 (UI consulta), RF13–RF22 (UI predicciones/billetera/ranking), RF26 (invitado). RNF01, RNF02, RNF03, RNF10.
Historias de usuario	HU01, HU02, HU03, HU04, HU05, HU07, HU08, HU09, HU11.
Procesos y subprocesos (AuraPortal)	Registro e inicio/cierre de sesión de usuario e invitado (RF01–RF03, RF26) · Consulta pública del torneo (calendario, posiciones, estadísticas, RF04–RF09) · Consulta de billetera y ranking (RF13, RF14, RF21) · Subproceso: Creación de una predicción desde la interfaz (RF15).
Diagramas UML (PowerDesigner)	Activity · Interaction Overview · Class · Object.
Interfaces Frontend (Balsamiq)	Diseña e implementa las mismas: registro/login, calendario del torneo, tabla de posiciones por grupo, estadísticas por selección, formulario de predicción 1X2, billetera/historial, ranking.
Capítulos del informe	1. Introducción (1.1 ¿Qué es?, 1.2 ¿Qué hace?) · 2.8 Visión General · 3.4 Historias de Usuario (redacción del apartado completo) · 3.5 Proceso (procesos del frontend público) · 4.2.3 Características de los usuarios · 4.2.3.3 Descripción de la aplicación.
Rol transversal / QA	Ejecuta los casos de prueba de la experiencia pública y apoya la integración de la Semana 6. Ejecuta CP04, CP05, CP12, CP14 desde la interfaz.
Entregables	Frontend Público funcional y temático · mockups Balsamiq · ADR del frontend JSF · Diagramas de actividad, visión de interacción, clases y objetos.
Dependencias	Depende de endpoints de Alexis (consulta) y Jonathan (predicciones/billetera/ranking). Recibe los contratos API antes de integrar (Sprints 4–5). Coordina el Use Case general con Jonathan y Fer.

6. Reparto de los 12 diagramas UML (PowerDesigner)

Los diagramas se reparten entre tres integrantes (Alexis queda excluido). El reparto no es por cantidad sino por complejidad y por coherencia con el módulo que cada uno desarrolla, buscando un esfuerzo equilibrado.

Integrante	Diagramas	N.º	Peso
Jonathan	Sequence, Communication, Statechart, Use Case	4	Alto
Fer	Deployment, Component, Package, Composite Structure	4	Alto
Dayana	Activity, Interaction Overview, Class, Object	4	Medio
Alexis	Ninguno	0	Compensado con backend + procesos + despliegue + QA

7. Proceso global integrador end-to-end (colaborativo)

El proceso «predecir → registrar resultado → liquidar premio → actualizar ranking» cruza los cuatro componentes. Se modela de forma colaborativa: cada integrante modela el tramo que le corresponde según su componente y luego se ensamblan en un único diagrama de proceso durante la Semana 4. Fer consolida el diagrama unificado en AuraPortal.

Paso	Tramo del proceso	Responsable
1	El usuario crea la predicción desde la interfaz pública	Dayana (Frontend Público)
2	UTNGolCoin valida saldo, cierra y almacena la predicción	Jonathan (Backend UTNGolCoin)
3	El administrador registra el resultado oficial desde el panel	Fer (Frontend Administrativo)
4	Estadísticas actualiza posiciones y notifica a UTNGolCoin	Alexis (Backend Estadísticas)
5	UTNGolCoin liquida premios y actualiza el saldo	Jonathan (Backend UTNGolCoin)
6	El ranking y la billetera se reflejan en la interfaz pública	Dayana (Frontend Público)

8. Balance de carga del equipo

Integrante	Diagramas	Procesos	Pantallas Balsamiq	Rol transversal extra
Alexis	0	6	0	Despliegue Linux (ambos backends), contrato REST común, RNF05, CP04/09/13, capítulos extra
Jonathan	4	5	0	CP de dominio UTNGolCoin
Fer	4	4 + consolidación E2E	6	Lidera integración y QA
Dayana	4	4	7	QA de experiencia pública

9. Cronograma de trabajo (6 sprints)

Semana	Objetivo	Actividades principales	Dependencias
1	Decidir y modelar	Requisitos (RF/RNF), historias de usuario, backlog. ADR de cada integrante (obligatorio). UML esenciales: Dayana (Class), Fer (Deployment/Package), Jonathan (Sequence base y Use Case). Alexis y Jonathan diseñan sus BD. Fer y Dayana: wireframes Balsamiq. Procesos iniciales en AuraPortal. Repos y entornos Linux listos.	Contratos REST preliminares acordados aquí (raíz).
2	Estadísticas	Alexis: API de Estadísticas, seed (48/12/104), endpoints de consulta, autenticación básica y Swagger. Jonathan avanza el modelo de UTNGolCoin. Fer y Dayana maquetan contra datos simulados.	Fer y Dayana dependen del contrato de Alexis.
3	UTNGolCoin y Administración	Jonathan: registro con bono de 10 (ledger), billetera y saldo. Fer: frontend admin (gestión de partidos, registro de resultados). Alexis expone el endpoint receptor de notificación. Cierran Statechart (Jonathan) y Component (Fer).	Fer y Jonathan dependen de la autenticación/identificador de Alexis.
4	Predicciones — los servicios conversan	Jonathan: predicciones (validación, cierre, única) + liquidación automática; Alexis dispara la notificación al registrar resultado. Sesión conjunta para el proceso end-to-end. Jonathan termina Sequence y Communication; Fer avanza Deployment con la topología final.	Dependencia crítica Alexis ↔ Jonathan (RF12/RF19).
5	Experiencia pública	Dayana completa el Frontend Público temático (calendario, estadísticas, posiciones, predicciones, ranking, bono diario). Cierra Activity, Interaction Overview y Object. Fer inicia pruebas de integración.	Dayana depende de endpoints estables de Alexis y Jonathan.
6	Cierre — calidad y entrega	Pruebas integrales (CP01–CP14), corrección de defectos, despliegue final en Linux (lidera Alexis), manuales técnico y de usuario, informe consolidado. Todos revisan sus capítulos y preparan la defensa.	Depende de todos los componentes terminados.